

Lab 5

name: WangShiyu nuist_id: 202383910020 setu_id: W20109917

1. Experiment Purpose

- 1) **Understand Istio Fundamentals:** Learn the core concepts of Istio service mesh and the roles of VirtualService, DestinationRule, and Gateway in traffic management.
- 2) **Implement Canary Deployments:** Demonstrate how to gradually shift traffic between microservice versions using weighted routing to minimize deployment risks.
- 3) **Configure Content-Based Routing:** Learn to route requests to specific service versions based on HTTP headers for A/B testing and personalized deployments.
- 4) **Test Resilience with Fault Injection:** Simulate network failures and service errors to validate microservices robustness and identify timeout misconfigurations.
- 5) **Protect Services with Circuit Breaking:** Configure connection limits and outlier detection to prevent cascading failures and maintain service stability.

2. Experiment Process

1) Environment Setup

(1) Infrastructure Verification

Check kubernetes cluster status.

```
PS C:\Users\WangShiyu> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:54880
CoreDNS is running at https://127.0.0.1:54880/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\Users\WangShiyu> kubectl get nodes
NAME        STATUS    ROLES    AGE     VERSION
minikube    Ready     control-plane 3d10h   v1.35.1
```

(2) Istio Installation Verification

Check control plane components. Core components, including istiod, istio-ingressgateway and istio-egressgateway, show STATUS: Running.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get pods -n istio-system
NAME                                READY   STATUS    RESTARTS   AGE
grafana-665dcbcb9b-jlhcb            0/1    ImagePullBackOff    0          51m
istio-egressgateway-5484c95fdb-5p49j 1/1    Running           0          77m
istio-ingressgateway-74c878c76-hld6s 1/1    Running           0          77m
istiod-599bd565bd-j4545             1/1    Running           0          77m
jaeger-75846c549c-44qz7             0/1    ImagePullBackOff    0          51m
kiali-55d986fccf-s62nk              1/1    Running           0          28m
loki-0                               0/2    ImagePullBackOff    0          51m
prometheus-788d9f4dbd-jhzz2         2/2    Running           0          37m
```

Check version consistency. All three version lines match, ensuring compatibility.

```
PS C:\Users\WangShiyu\istio-1.28.0> istioctl version
client version: 1.28.0
control plane version: 1.28.0
data plane version: 1.28.0 (8 proxies)
```

(3) Sidecar Injection Verification

The label istio-injection=enabled confirms automatic sidecar injection is active.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get namespace default --show-labels
NAME        STATUS    AGE     LABELS
default    Active    3d11h   istio-injection=enabled,kubernetes.io/metadata.name=default
```

(4) Bookinfo Deployment Verification

Check service status.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get services
NAME          TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)    AGE
details       ClusterIP   10.101.18.210 <none>        9080/TCP    63m
kubernetes    ClusterIP   10.96.0.1     <none>        443/TCP     78m
productpage   ClusterIP   10.103.108.4  <none>        9080/TCP    63m
ratings       ClusterIP   10.108.207.141 <none>        9080/TCP    63m
reviews       ClusterIP   10.110.72.29  <none>        9080/TCP    63m
```

Check pod status.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-6d999b86f-wz2sz         2/2     Running   0           63m
productpage-v1-6fd84d55f6-lhpxv    2/2     Running   0           63m
ratings-v1-784cc59f6f-tqkqf       2/2     Running   0           63m
reviews-v1-59484bb9d5-nt8mz       2/2     Running   0           63m
reviews-v2-8965c56b7-kvgn9        2/2     Running   0           63m
reviews-v3-65668586c7-vw9rl       2/2     Running   0           63m
```

(5) Gateway Configuration

Check Gateway Configuration.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get gateways.networking.istio.io
NAME          AGE
bookinfo-gateway 41m
```

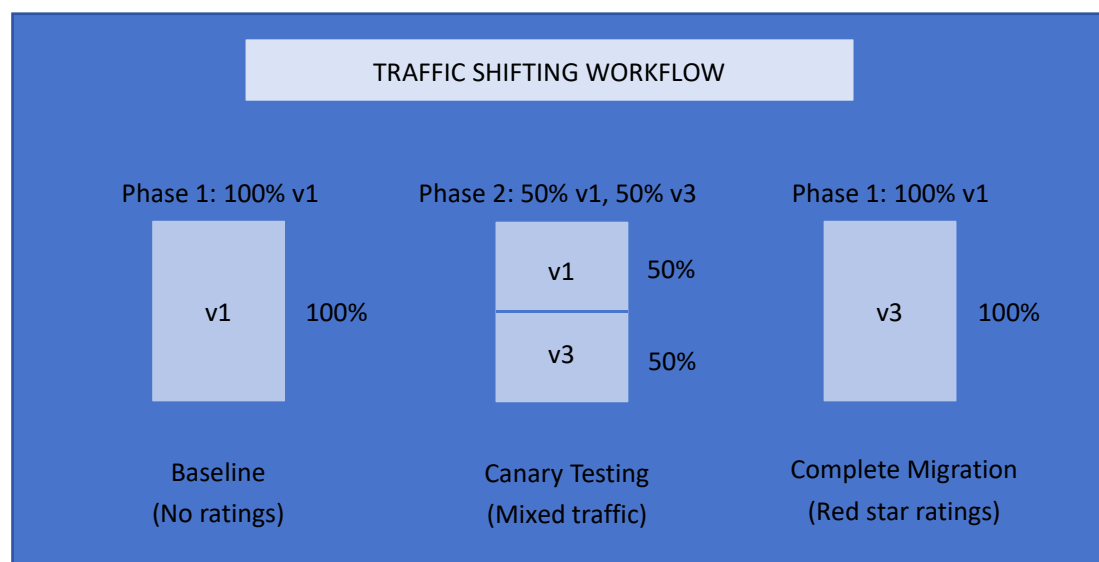
Check VirtualService Configuration.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get virtualservice
NAME          GATEWAYS          HOSTS          AGE
bookinfo      ["bookinfo-gateway"] ["*"]          61m
```

2) Experiment 1: Traffic Shifting

Demonstrate how to gradually shift traffic from one version of a microservice to another, implementing a canary deployment pattern .

Traffic Shifting Process Flow:



(1) Step 1: Configure Destination Rules

Apply destination rules (define service subsets).

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/destination-rule-all.yaml
destinationrule.networking.istio.io/productpage created
destinationrule.networking.istio.io/reviews created
destinationrule.networking.istio.io/ratings created
destinationrule.networking.istio.io/details created
```

Four destination rules are listed, confirming subset definitions exist for all services.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get destinationrules
NAME      HOST      AGE
details   details   11h
productpage productpage 11h
ratings   ratings   11h
reviews   reviews   11h
```

(2) Step 2: Route All Traffic to v1

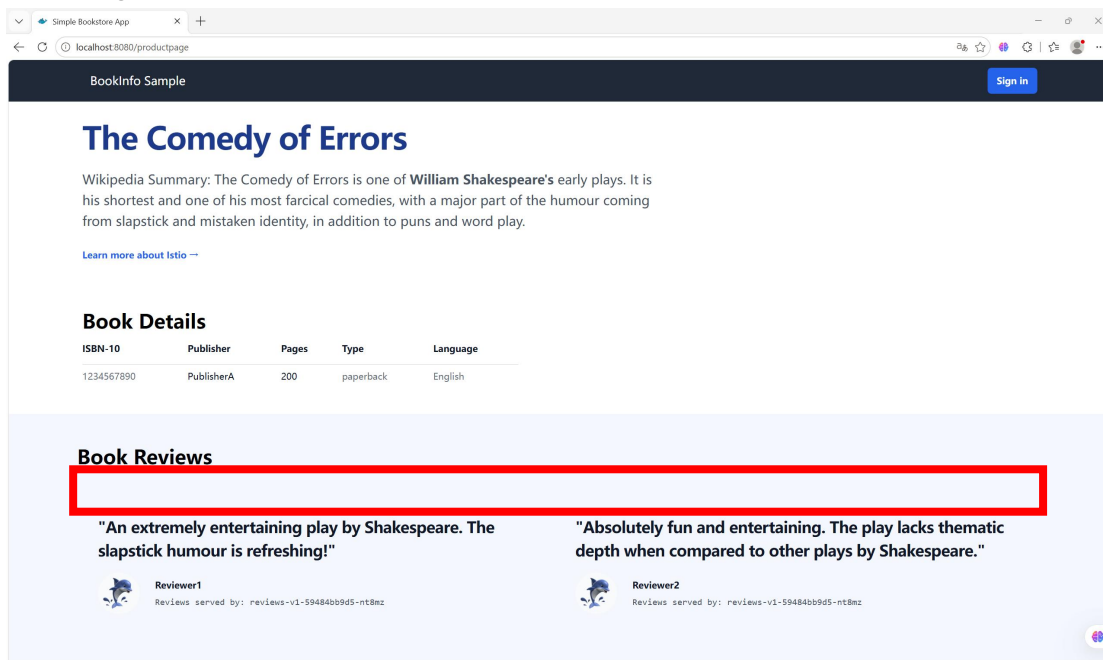
Route all traffic to v1 version.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage created
virtualservice.networking.istio.io/reviews created
virtualservice.networking.istio.io/ratings created
virtualservice.networking.istio.io/details created
```

Four new VirtualServices created with HOSTS matching service names.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get virtualservice
NAME      GATEWAYS      HOSTS      AGE
bookinfo  ["bookinfo-gateway"] ["*"]      11h
details   ["details"]    3m4s
productpage ["productpage"] 3m4s
ratings   ["ratings"]    3m4s
reviews   ["reviews"]    3m4s
```

Browser Testing: Multiple page refreshes showed no star ratings in the reviews section, confirming all traffic routes to reviews:v1.



(3) Step 3: Shift 50% Traffic to v3

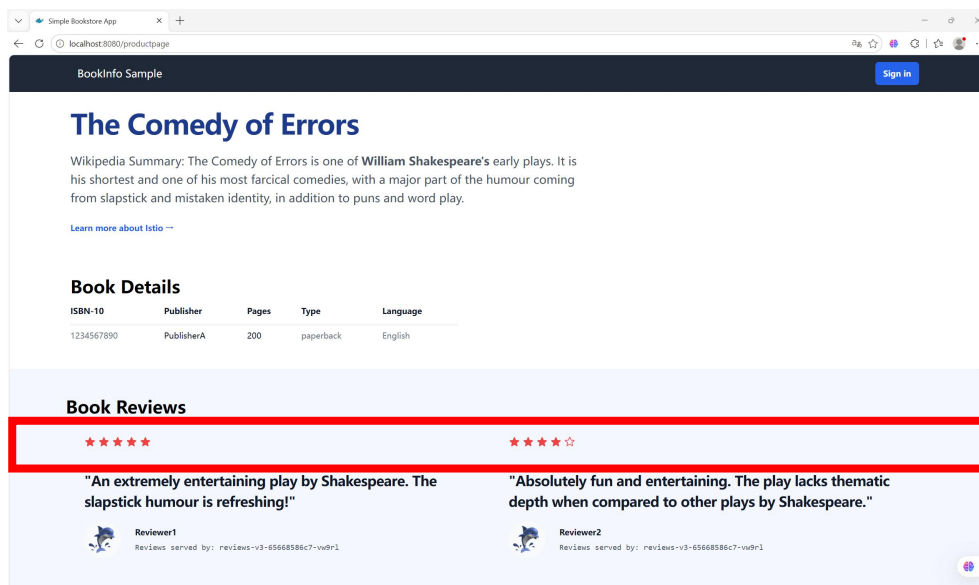
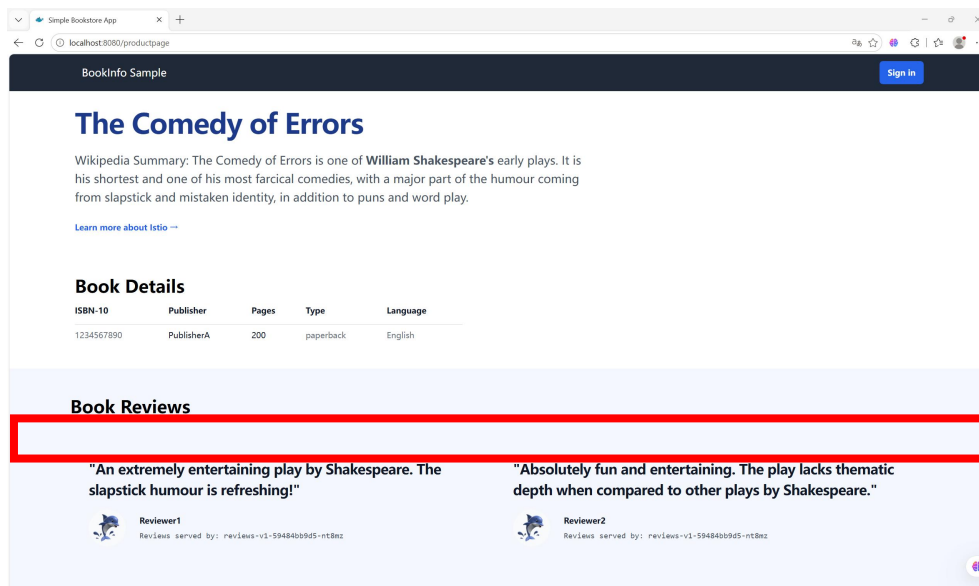
The weight: 50 for both v1 and v3 subsets confirms equal traffic distribution.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-50-v3.yaml
virtualservice.networking.istio.io/reviews configured
```

Verify configuration.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","name
space":"default"},"spec":{"hosts":["reviews"],"http":[{"route":[{"destination":{"host":"reviews","subset":"v1"},"weight"
:50},{"destination":{"host":"reviews","subset":"v3"},"weight":50}]}]}
creationTimestamp: "2026-05-16T00:44:54Z"
generation: 2
name: reviews
namespace: default
resourceVersion: "470930"
uid: c80bc064-d22c-446c-b6a0-788834c79e48
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
        weight: 50 ← 50% to v1
    - destination:
        host: reviews
        subset: v3
        weight: 50 ← 50% to v3
```

Browser Testing: Approximately half of the page refreshes displayed red star ratings (v3), while the other half showed no ratings (v1).



(4) Step 4: Complete Migration to v3

Migrate all traffic to v3.

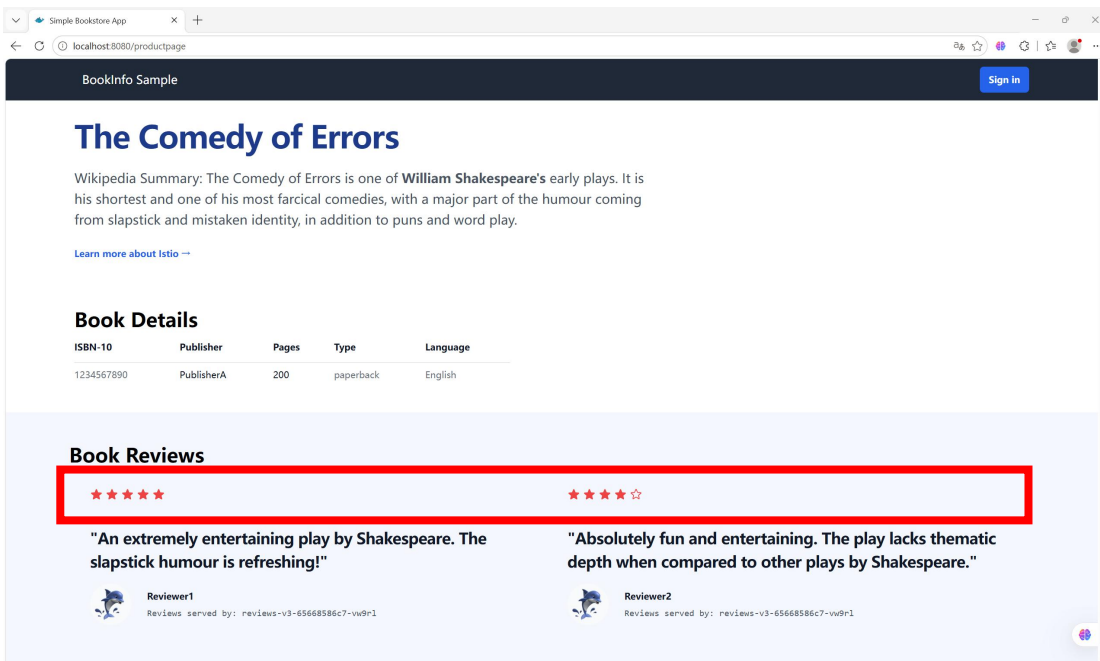
```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-v3.yaml
virtualservice.networking.istio.io/reviews configured
```

Only one destination is listed with subset: v3, indicating 100% traffic to v3.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","namespace":"default"},"spec":{"hosts":["reviews"],"http":[{"route":[{"destination":{"host":"reviews","subset":"v3"}}]}]}}
  creationTimestamp: "2026-05-16T00:44:54Z"
  generation: 3
  name: reviews
  namespace: default
  resourceVersion: "477003"
  uid: c80bc064-d22c-446c-b6a0-788834c79e48
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v3
```

← Only v3, no weight specified = 100%

Browser Testing: All page refreshes consistently displayed red star ratings.



(5) Step 5: Analysis and Results

Traffic Distribution Results:

Phase	v1 Weight	v3 Weight	Observed Behavior
Phase 1	100%	0%	No star ratings in all requests
Phase 2	50%	50%	Mixed: approximately half with red stars, half without
Phase 3	0%	100%	Red star ratings in all requests

Technical Implementation Details:

The traffic shifting mechanism works through Envoy proxy configuration:

- VirtualService defines routing rules with weights
- istiod (control plane) translates rules into Envoy configuration

- Envoy sidecars distribute requests based on weighted round-robin

The weight distribution is probabilistic at the request level, not deterministic. Therefore, the observed distribution in a small sample may slightly deviate from the configured weights, but it statistically converges to the configured values over many requests.

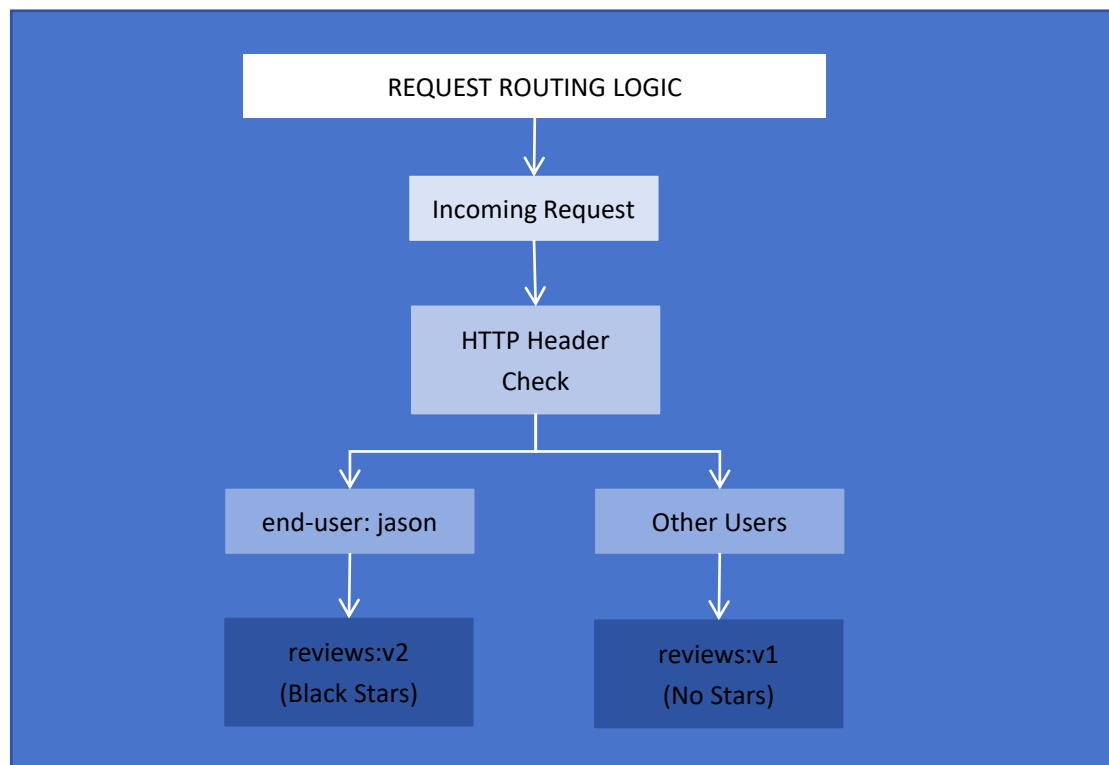
(6) Step 6: Cleanup

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl delete -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io "productpage" deleted
virtualservice.networking.istio.io "reviews" deleted
virtualservice.networking.istio.io "ratings" deleted
virtualservice.networking.istio.io "details" deleted
```

3) Experiment 2: Request Routing

Demonstrate content-based routing where requests are directed to different service versions based on HTTP headers, enabling A/B testing and personalized deployments .

Request Routing Decision Flow:



(1) Step 1: Configure Default Routing

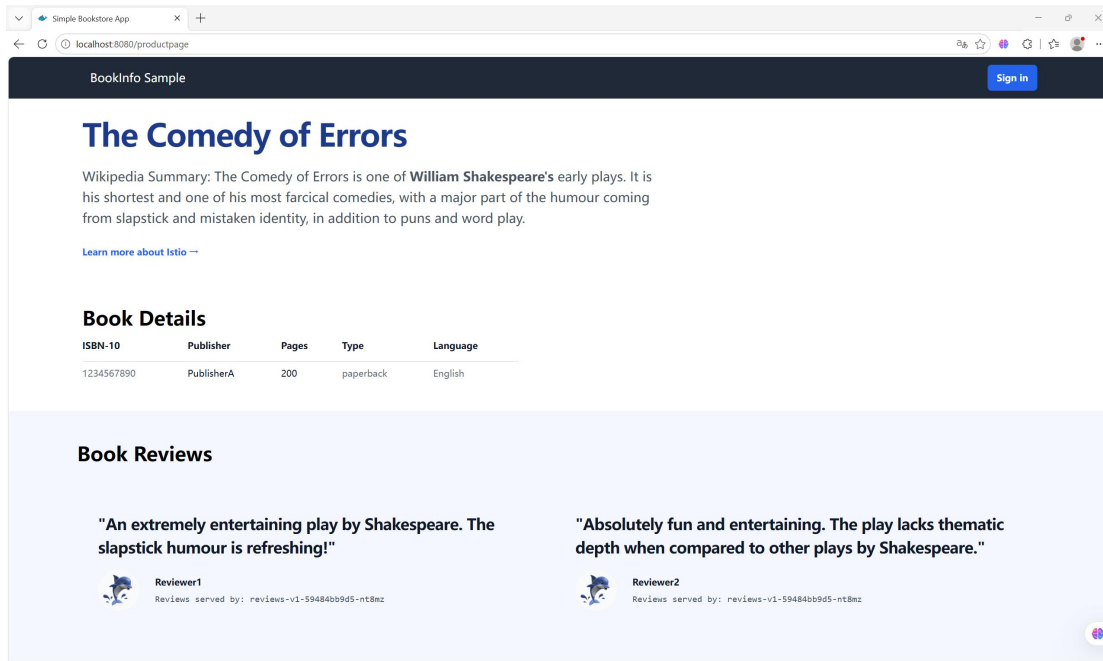
Configure Default Routing to v1:

- Apply destination rules
- Route all traffic to v1
- Verify routing rules

All VirtualServices are listed with AGE: 7s, indicating fresh creation. The reviews:v1 does not call ratings service, so no ratings are displayed.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/destination-rule-all.yaml
destinationrule.networking.istio.io/productpage unchanged
destinationrule.networking.istio.io/reviews unchanged
destinationrule.networking.istio.io/ratings unchanged
destinationrule.networking.istio.io/details unchanged
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage created
virtualservice.networking.istio.io/reviews created
virtualservice.networking.istio.io/ratings created
virtualservice.networking.istio.io/details created
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get virtualservices
NAME          GATEWAYS          HOSTS          AGE
bookinfo      ["bookinfo-gateway"] ["*"]          12h
details       ["details"]        7s
productpage   ["productpage"]    7s
ratings       ["ratings"]         7s
reviews       ["reviews"]         7s
```

Browser Testing: Page refreshes showed no star ratings, confirming default routing to v1.



(2) Step 2: Configure Header-Based Routing

Apply user-based routing rules.

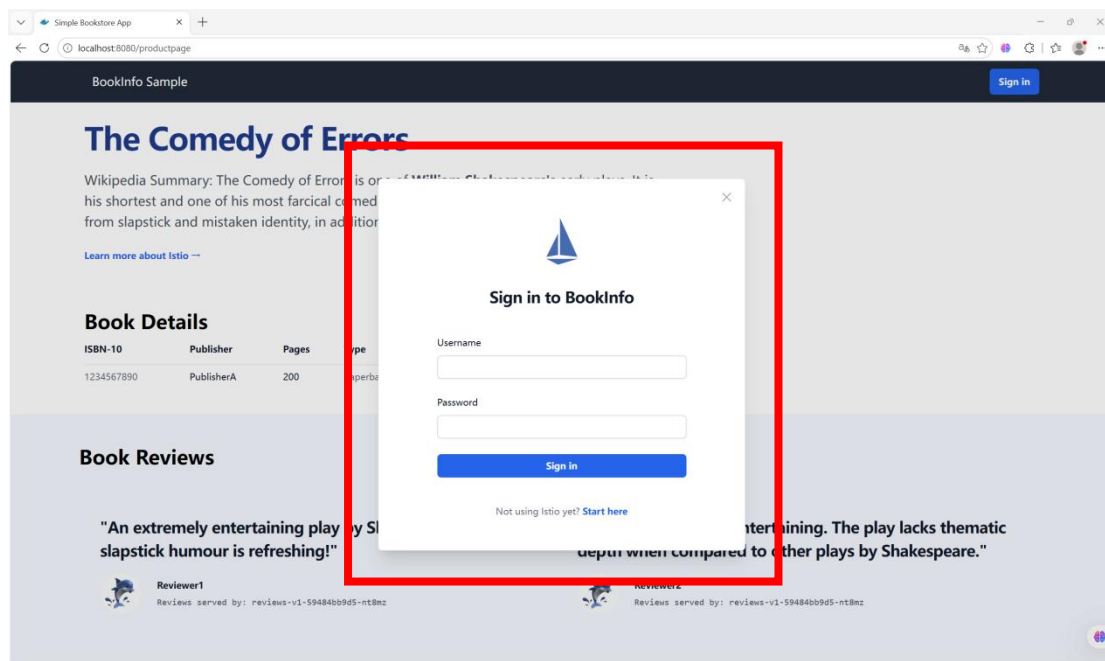
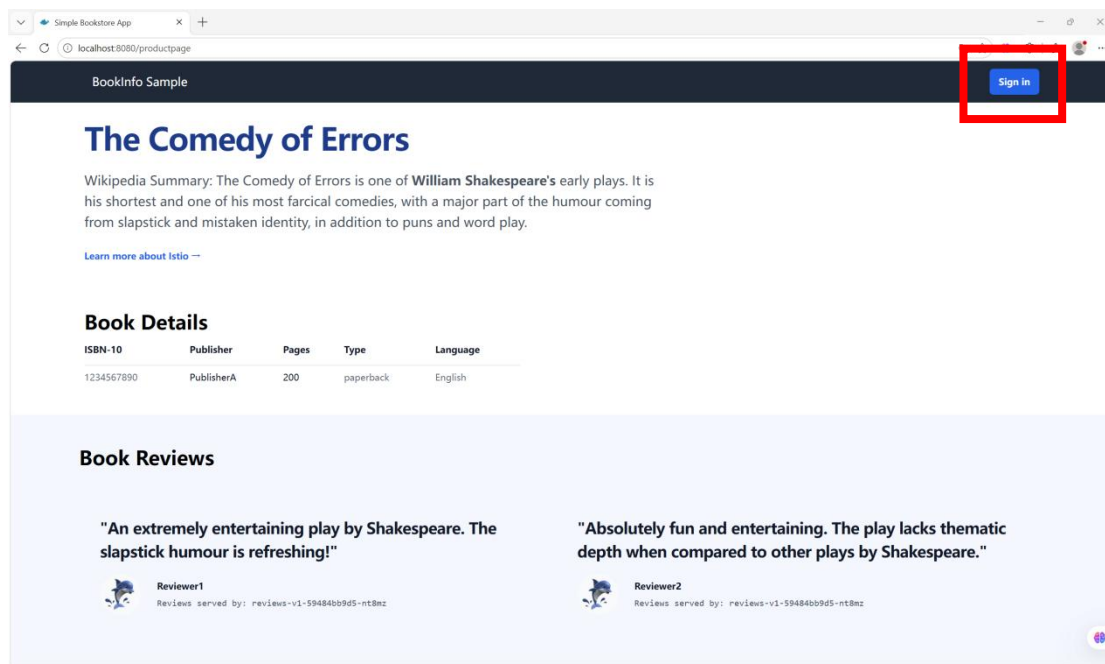
```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml
virtualservice.networking.istio.io/reviews configured
```

The match section with headers.end-user.exact: jason confirms header-based routing logic.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{"name":"reviews","name
space":"default"},"spec":{"hosts":["reviews"],"http":[{"match":[{"headers":{"end-user":{"exact":"jason"}}}],route":[{"d
estination":{"host":"reviews","subset":"v2"}}]},{"route":[{"destination":{"host":"reviews","subset":"v1"}}]}}}
    creationTimestamp: "2026-05-16T02:26:05Z"
  generation: 2
  name: reviews
  namespace: default
  resourceVersion: "478948"
  uid: 407b0de0-160d-4572-8ba7-8c3e8c15ed20
spec:
  hosts:
  - reviews
  http:
  - match:
    - headers:
        end-user:
          exact: jason
      route:
        - destination:
            host: reviews
            subset: v2
    - route:
        - destination:
            host: reviews
            subset: v1
```

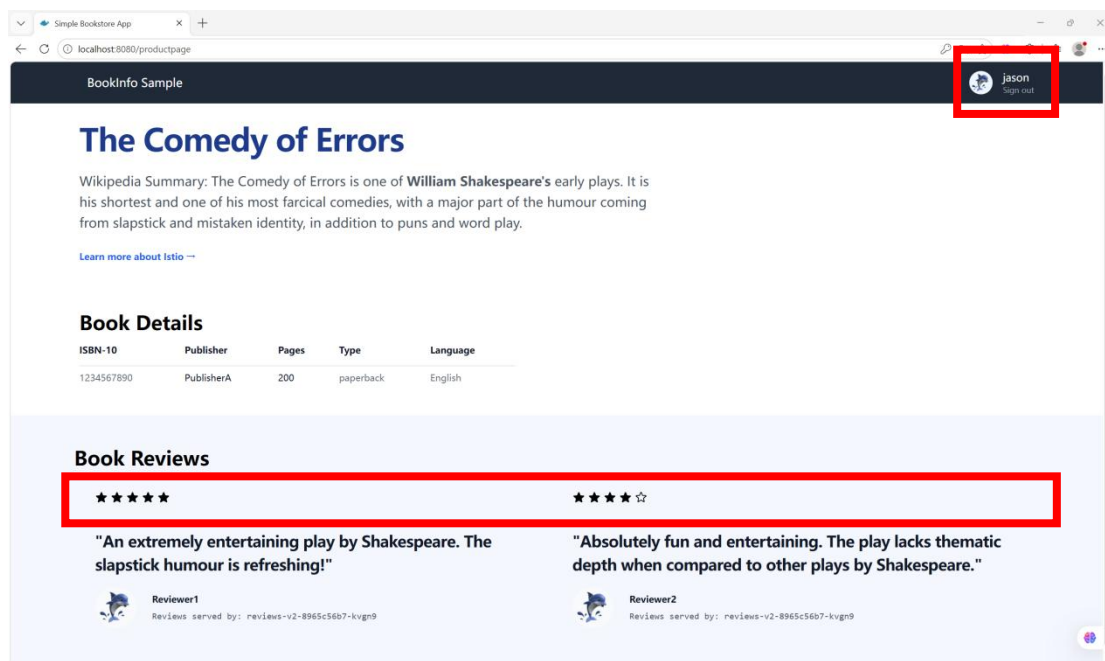
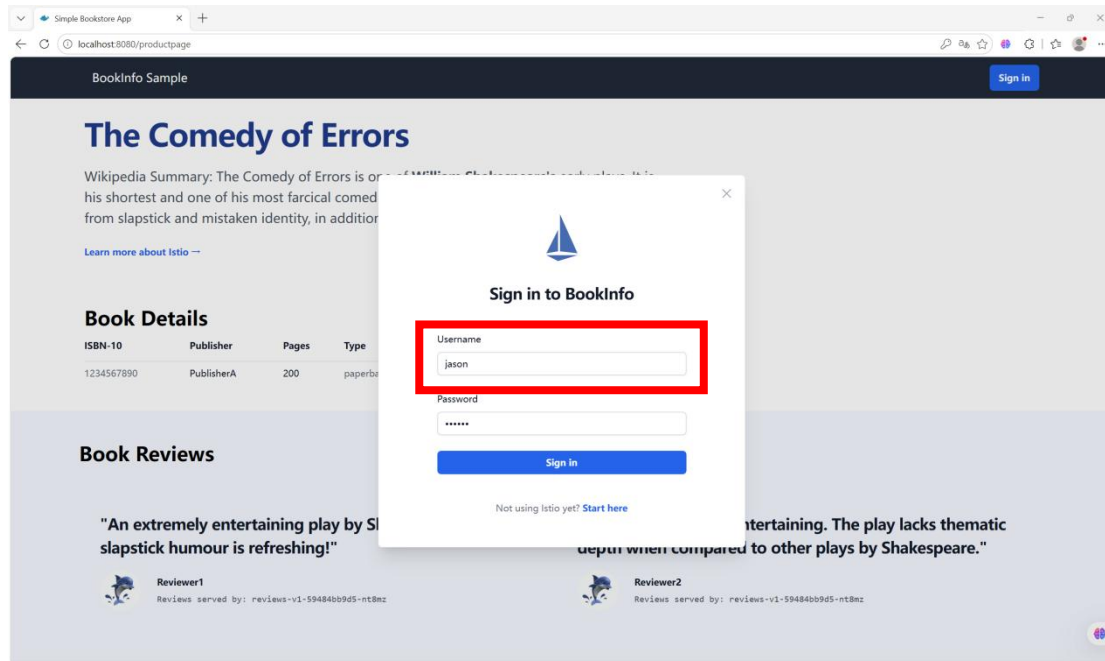
(3) Step 3: Testing and Results

Open <http://localhost:8080/productpage> in browser, and click "Sign in" in the top right corner.



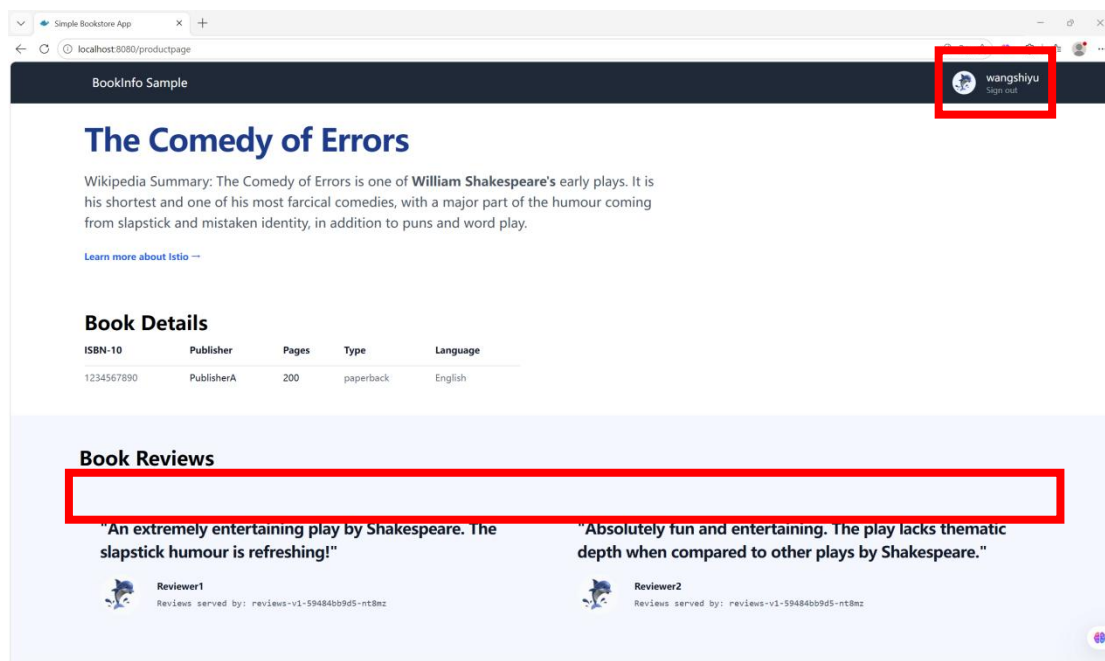
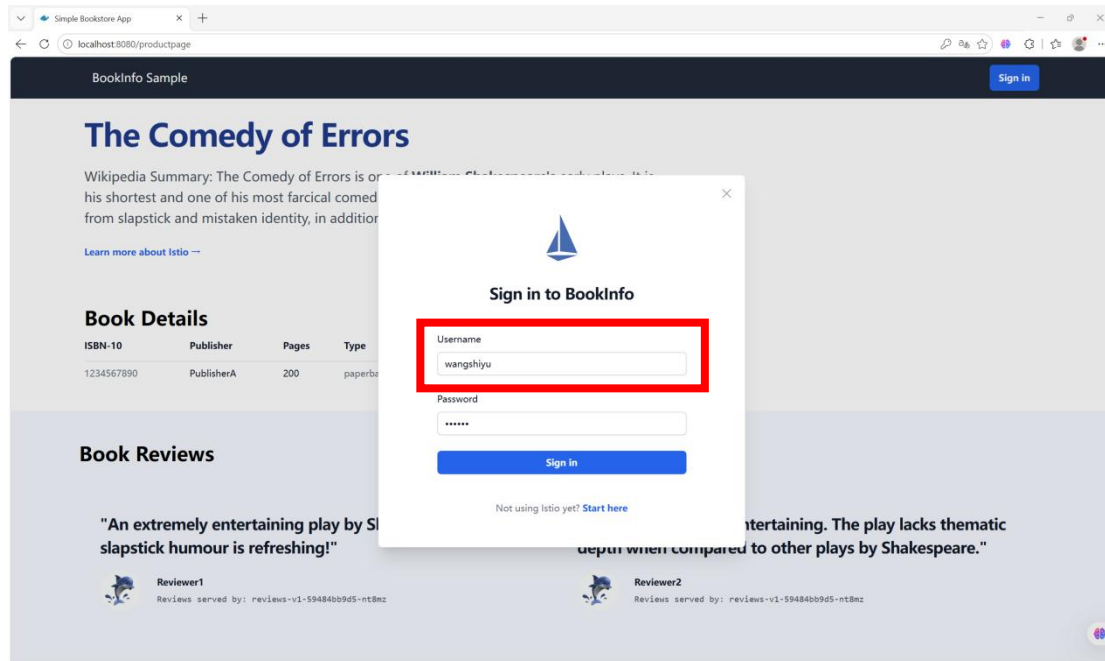
Test Case A: User "jason":

- Action: Login with username jason (password arbitrary)
- Expected: Route to reviews:v2 (black stars)
- Actual: Black star ratings displayed
- Evidence: Browser screenshot captured



Test Case B: Other Users

- Action: Login with username other than jason
- Expected: Route to reviews:v1 (no stars)
- Actual: No star ratings displayed
- Evidence: Browser screenshot captured

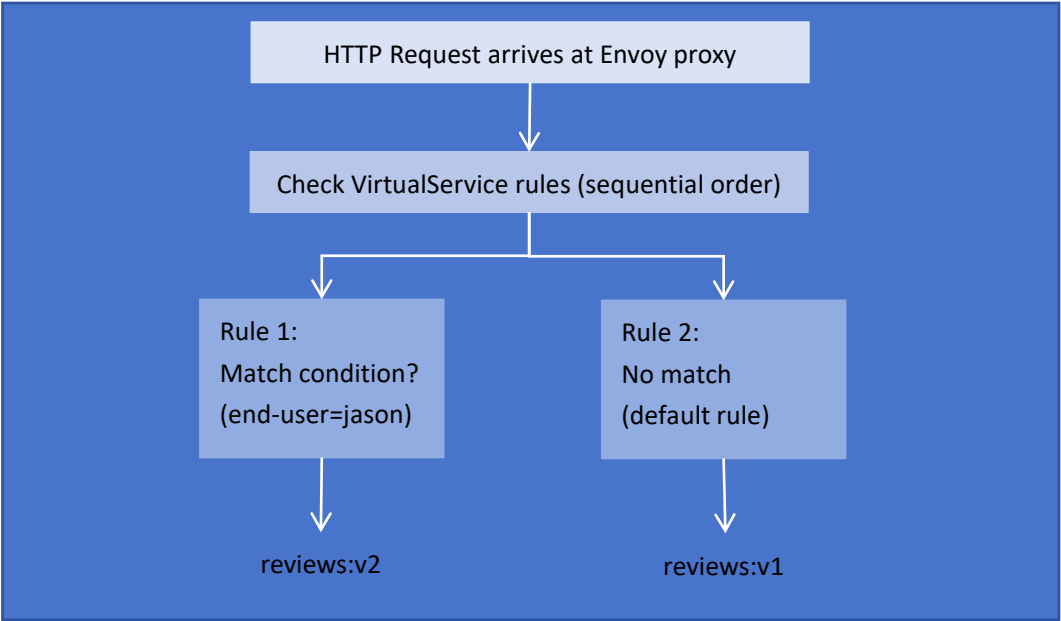


(4) Step 4: Analysis

Routing Rule Evaluation Order:

Istio evaluates routing rules in the order they appear in the VirtualService specification. The first

matching rule is applied. If no match conditions exist (like the second route), it serves as a default catch-all rule.



(5) Step 5: Cleanup

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl delete -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io "productpage" deleted
virtualservice.networking.istio.io "reviews" deleted
virtualservice.networking.istio.io "ratings" deleted
virtualservice.networking.istio.io "details" deleted
```

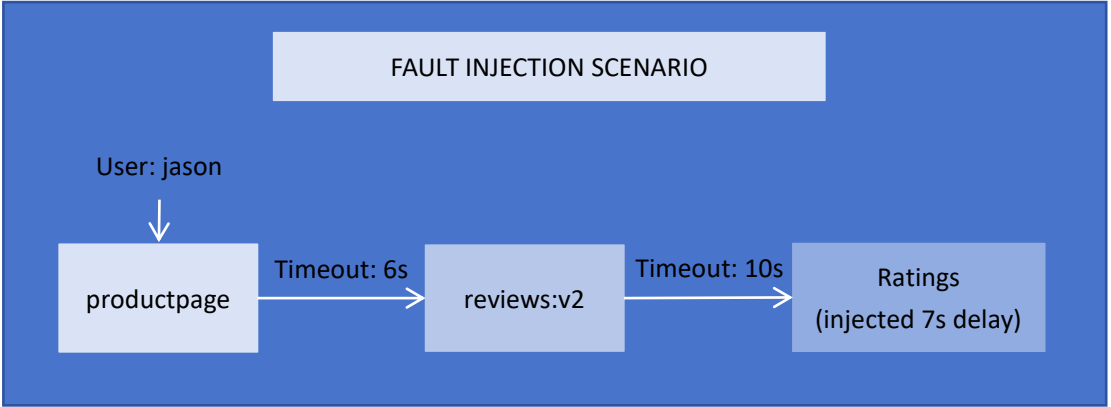
4) Experiment 3: Fault Injection

Demonstrate fault injection to test microservices resilience by simulating network failures and service errors .

Fault Injection Types:

Type	Configuration	Effect	Use Case
Delay	fixedDelay: 7s	Adds latency to requests	Test timeout handling
Abort	httpStatus: 500	Returns error response	Test error recovery logic

Fault Injection Architecture:



(1) Step 1: Establish Baseline Routing

Configuration Logic:

- User jason → reviews:v2 → ratings:v1
- Other users → reviews:v1 (no ratings call)

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage created
virtualservice.networking.istio.io/reviews created
virtualservice.networking.istio.io/ratings created
virtualservice.networking.istio.io/details created
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml
virtualservice.networking.istio.io/reviews configured
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get virtualservices
NAME          GATEWAYS          HOSTS          AGE
bookinfo      ["bookinfo-gateway"] ["*"]          22h
details       ["details"]        17s
productpage   ["productpage"]    17s
ratings       ["ratings"]         17s
reviews       ["reviews"]         17s
```

(2) Step 2: Inject HTTP Delay Fault

Inject 7-second delay for jason user.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-delay.yaml
virtualservice.networking.istio.io/ratings configured
```

The fault.delay.fixedDelay: 7s with percentage.value: 100 confirms 7-second delay injection for all jason's requests.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get virtualservice ratings -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"ratings","namespace":"default"},"spec":{"hosts":["ratings"],"http":[{"fault":{"delay":{"fixedDelay":"7s","percentage":{"value":100}}},"match":[{"headers":{"end-user":{"exact":"jason"}}},"route":[{"destination":{"host":"ratings","subset":"v1"}}]}]}]}
  creationTimestamp: "2026-05-16T12:29:25Z"
  generation: 2
  name: ratings
  namespace: default
  resourceVersion: "534067"
  uid: 6cb70145-34b8-4d50-abfd-da969421a6f9
spec:
  hosts:
  - ratings
  http:
  - fault:
      delay:
        fixedDelay: 7s ← Inject 7 second delay
        percentage:
          value: 100 ← Apply to 100% of requests
    match:
    - headers:
        end-user:
          exact: jason ← Only for jason user
    route:
    - destination:
        host: ratings
        subset: v1
    - route:
        destination:
          host: ratings
          subset: v1
```

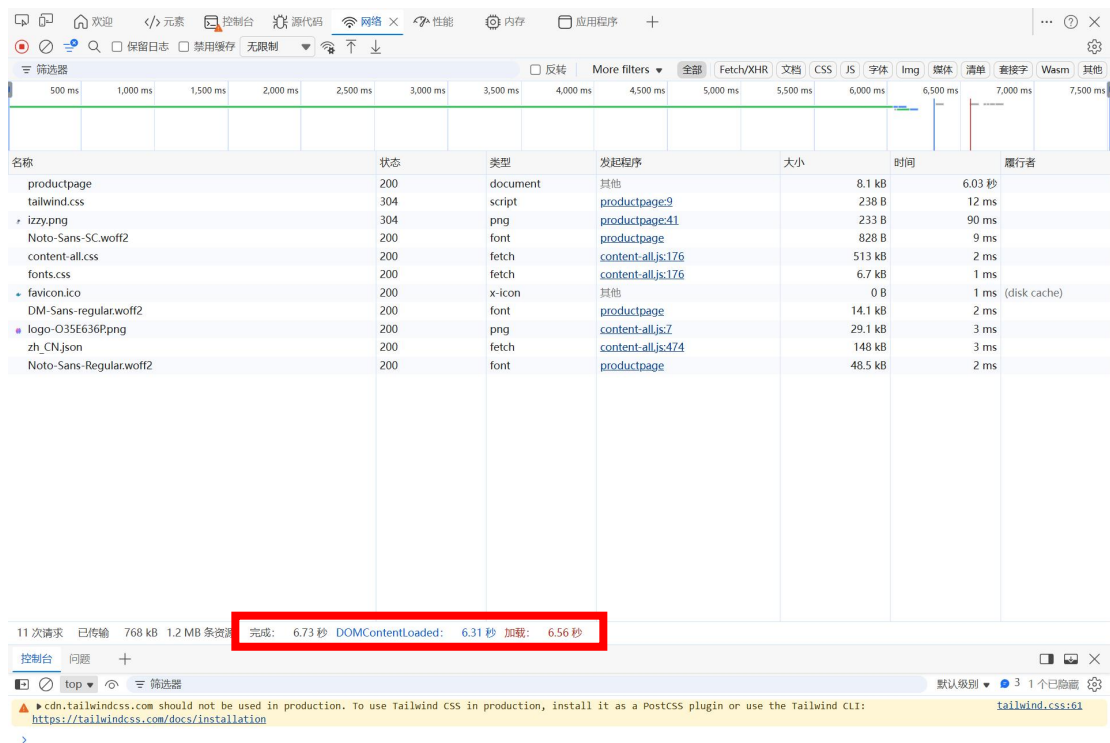
(3) Step 3: Test Delay Fault

Test Procedure:

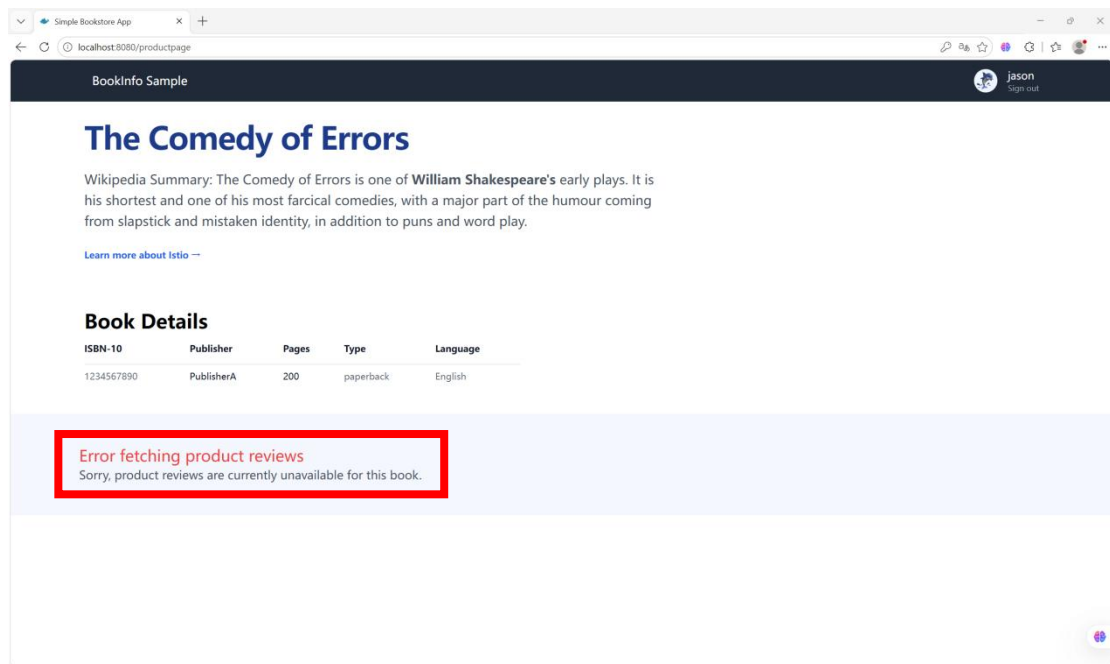
- Login as user jason
- Open browser Developer Tools (F12) → Network tab
- Refresh productpage
- Observe response time and page content

Actual Result:

Page should load with error message after approximately 6 seconds.



Error message: "Sorry, product reviews are currently unavailable for this book."



(4) Step 4: Analysis

Root Cause Analysis:

The experiment reveals a critical timeout configuration issue.

Service	Timeout Configuration	Implication
ratings	7s delay (injected)	Response delayed by 7 seconds

reviews	10s timeout	Should be sufficient (7s < 10s)
productpage	3s timeout + 1 retry	Total: 6 seconds maximum

Problem:

- productpage's 6-second timeout triggers before reviews can receive ratings response (7s delay)
- This exposes a microservices anti-pattern: cascading timeout misconfiguration

Architecture Flow with Fault:

Request Timeline (with 7s delay injected)
● 0s: productpage receives request ● 0.1s: productpage calls reviews ● 0.2s: reviews calls ratings ● 0.2s: ratings DELAYED (fault injection) ● 6s: productpage TIMEOUT (3s + 1 retry) ● 7.2s: ratings response ready (too late!)

(5) Step 5: Inject HTTP Abort Fault

Delete delay fault.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl delete -f samples/bookinfo/networking/virtual-service-ratings-test-delay.yaml
virtualservice.networking.istio.io "ratings" deleted
```

Inject HTTP 500 error.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-abort.yaml
virtualservice.networking.istio.io/ratings created
```

The fault.abort.httpStatus: 500 confirms HTTP 500 error injection.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get virtualservice ratings -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"ratings","namespace":"default"},"spec":{"hosts":["ratings"],"http":[{"fault":{"abort":{"httpStatus":500,"percentage":{"value":100}}},"match":[{"headers":{"end-user":{"exact":"jason"}}},"route":[{"destination":{"host":"ratings","subset":"v1"}}]}]}]}
      creationTimestamp: "2026-05-16T12:49:57Z"
  generation: 1
  name: ratings
  namespace: default
  resourceVersion: "535349"
  uid: 2c996643-625e-483e-96dd-4e57bb30cff7
spec:
  hosts:
  - ratings
  http:
  - fault:
      abort:
        httpStatus: 500
        percentage:
          value: 100
      match:
      - headers:
          end-user:
            exact: jason
        route:
        - destination:
            host: ratings
            subset: v1
      - route:
        - destination:
            host: ratings
            subset: v1
```

(6) Step 6: Test Abort Fault

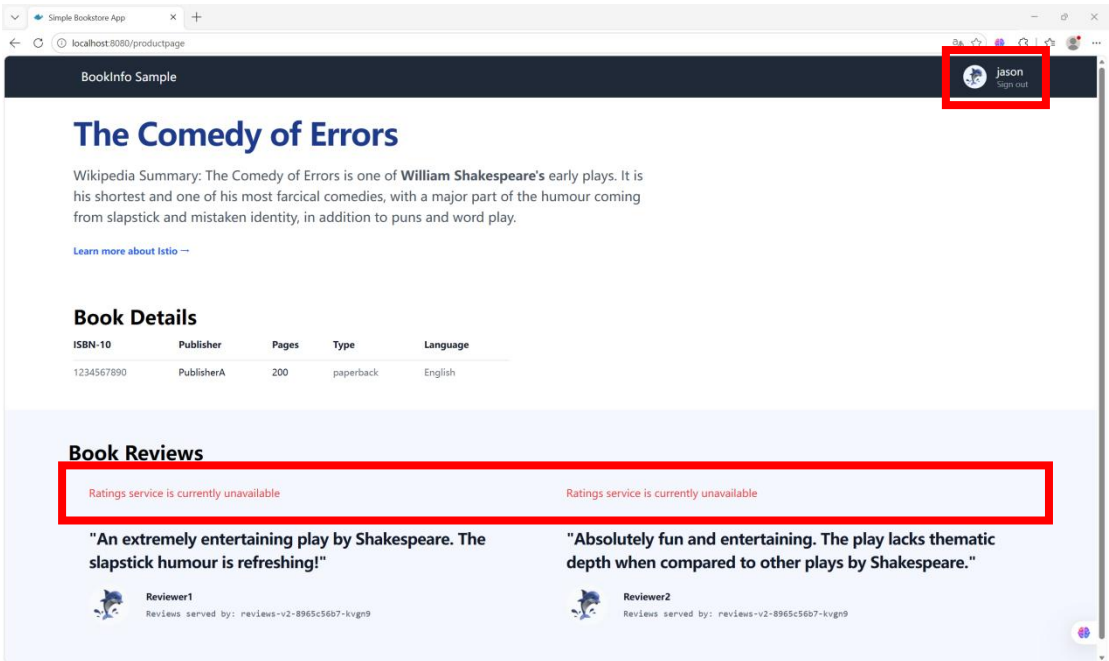
Test Procedure:

- Login as user jason
- Refresh productpage

- Observe immediate response

Actual Result:

- Page loads immediately (no delay)
- Error message: "Ratings service is currently unavailable"



Comparative Analysis: Delay vs Abort:

Aspect	Delay Fault	Abort Fault
Response Time	Delayed (waits for timeout)	Immediate
Resource Consumption	High (connections held open)	Low (quick failure)
Testing Purpose	Timeout and retry logic	Error handling and fallbacks
User Experience	Long wait before error	Quick error feedback

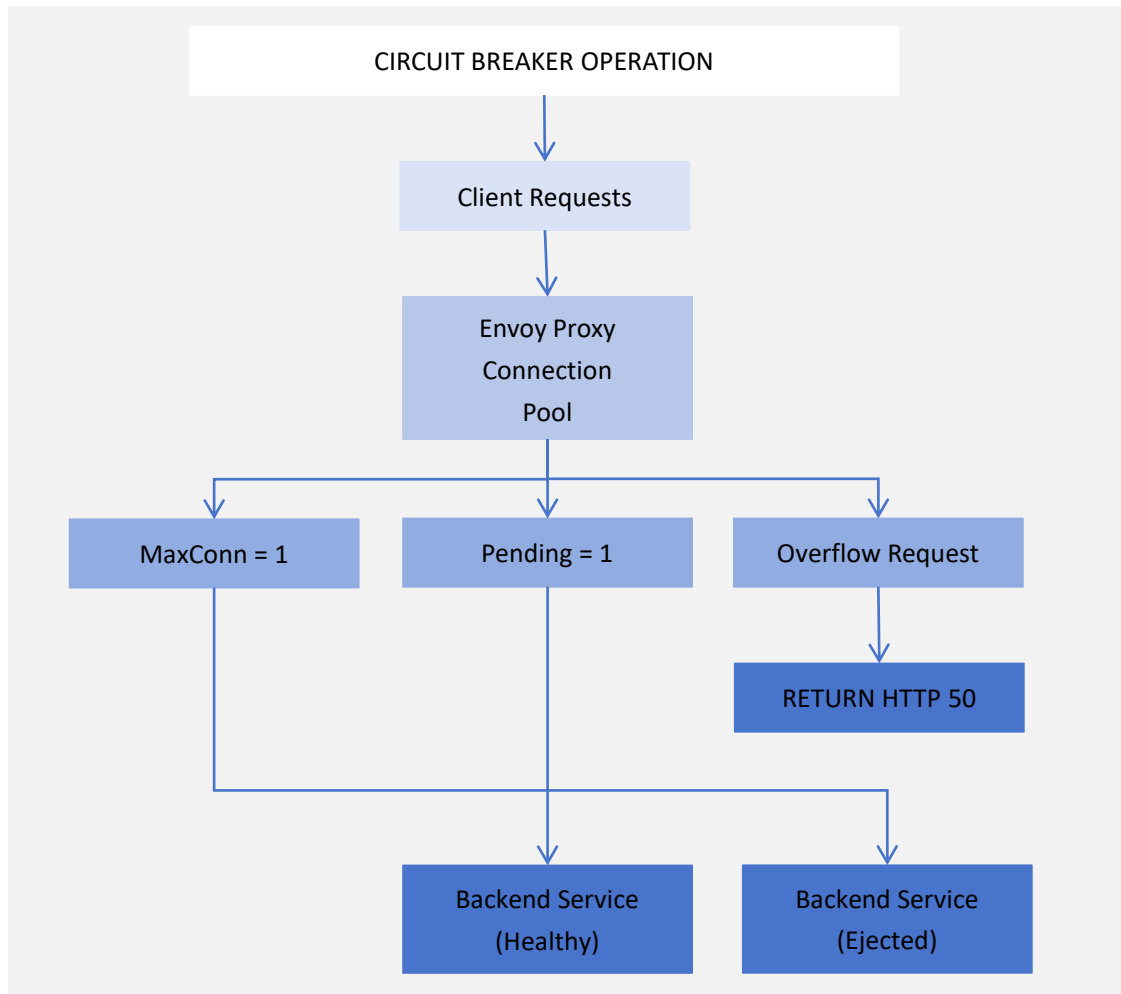
(7) Step 7: Cleanup

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl delete -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io "productpage" deleted
virtualservice.networking.istio.io "reviews" deleted
virtualservice.networking.istio.io "ratings" deleted
virtualservice.networking.istio.io "details" deleted
```

5) Experiment 4: Circuit Breaking

Configure and test circuit breaking to protect services from cascading failures by limiting concurrent connections and requests .

Circuit Breaking Mechanism:



(1) Step 1: Deploy Test Application

Pull and tag docker image (for China mainland network) to deploy httpbin service.

```

PS C:\Users\WangShiyu\istio-1.28.0> & minikube -p minikube docker-env --shell powershell | Invoke-Expression
PS C:\Users\WangShiyu\istio-1.28.0> docker pull hub.rat.dev/mccutchen/go-httpbin:v2.15.0
v2.15.0: Pulling from mccutchen/go-httpbin
a85ce7502e1a: Pull complete
e8d9a567199d: Pull complete
058cf3d8c2ba: Pull complete
b6824ed73363: Pull complete
7c12895b777b: Pull complete
33e068de2649: Pull complete
5664b15f108b: Pull complete
27be814a09eb: Pull complete
4aa0ea1413d3: Pull complete
da7816fa955e: Pull complete
9aee425378d2: Pull complete
209150bce63e: Pull complete
903010886edf: Pull complete
c8e2e680d177: Pull complete
Digest: sha256:24528cf5229d0b70065ac27e6c9e4d96f5452a84a3ce4433e56573c18d96827a
Status: Downloaded newer image for hub.rat.dev/mccutchen/go-httpbin:v2.15.0
hub.rat.dev/mccutchen/go-httpbin:v2.15.0
PS C:\Users\WangShiyu\istio-1.28.0> docker tag hub.rat.dev/mccutchen/go-httpbin:v2.15.0 docker.io/mccutchen/go-httpbin:v
2.15.0
  
```

Pod status Running with READY: 2/2 confirms successful deployment with sidecar.

```

PS C:\Users\WangShiyu\istio-1.28.0> kubectl get pods -l app=httpbin
NAME                READY   STATUS    RESTARTS   AGE
httpbin-5695765667-ptd2r  2/2     Running   0           35s
  
```

(2) Step 2: Configure Circuit Breaker

Create DestinationRule to configure circuit breaking policy.

```

PS C:\Users\WangShiyu\istio-1.28.0> @
>> apiVersion: networking.istio.io/v1
>> kind: DestinationRule
>> metadata:
>>   name: httpbin
>> spec:
>>   host: httpbin
>>   trafficPolicy:
>>     connectionPool:
>>       tcp:
>>         maxConnections: 1
>>       http:
>>         http1MaxPendingRequests: 1
>>         maxRequestsPerConnection: 1
>>     outlierDetection:
>>       consecutive5xxErrors: 1
>>       interval: 1s
>>       baseEjectionTime: 3m
>>       maxEjectionPercent: 100
>> "@ | kubectl apply -f -
destinationrule.networking.istio.io/httpbin created

```

All parameters appear in the YAML output as configured.

```

PS C:\Users\WangShiyu\istio-1.28.0> kubectl get destinationrule httpbin -o yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"DestinationRule","metadata":{"annotations":{},"name":"httpbin","namespace":"default"},"spec":{"host":"httpbin","trafficPolicy":{"connectionPool":{"http":{"http1MaxPendingRequests":1,"maxRequestsPerConnection":1},"tcp":{"maxConnections":1}},"outlierDetection":{"baseEjectionTime":"3m","consecutive5xxErrors":1,"interval":"1s","maxEjectionPercent":100}}}}
  creationTimestamp: "2026-05-16T13:16:01Z"
  generation: 1
  name: httpbin
  namespace: default
  resourceVersion: "537941"
  uid: 9407c19f-2e2f-4aa7-b53e-119ad7885056
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
      tcp:
        maxConnections: 1
    outlierDetection:
      baseEjectionTime: 3m
      consecutive5xxErrors: 1
      interval: 1s
      maxEjectionPercent: 100

```

Key Configuration Parameters:

Parameter	Value	Purpose
maxConnections	1	Maximum TCP connections to backend
http1MaxPendingRequests	1	Maximum requests waiting in queue
maxRequestsPerConnection	1	Requests per connection before close
consecutive5xxErrors	1	Errors before ejection
interval	1s	Detection interval
baseEjectionTime	3m	Minimum ejection duration
maxEjectionPercent	100	Maximum hosts to eject

(3) Step 3: Deploy Load Testing Client

Deploy fortio load testing tool

```

PS C:\Users\WangShiyu\istio-1.28.0> kubectl apply -f samples/httpbin/sample-client/fortio-deploy.yaml
service/fortio created
deployment.apps/fortio-deploy created

PS C:\Users\WangShiyu\istio-1.28.0> & minikube -p minikube docker-env --shell powershell | Invoke-Expression
PS C:\Users\WangShiyu\istio-1.28.0> docker pull hub.rat.dev/fortio/fortio:latest_release
latest_release: Pulling from fortio/fortio
a97b19e89962: Pull complete
2d974c5cb5c4: Pull complete
Digest: sha256:fc8221136fe281d733e8e42c07440fa750848a72fcd835726b509088d1f5264
Status: Downloaded newer image for hub.rat.dev/fortio/fortio:latest_release
hub.rat.dev/fortio/fortio:latest_release
PS C:\Users\WangShiyu\istio-1.28.0> docker tag hub.rat.dev/fortio/fortio:latest_release fortio/fortio:latest_release

```

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl set image deployment/fortio-deploy fortio=hub.rat.dev/fortio/fortio:latest_release
deployment.apps/fortio-deploy image updated
```

Wait for fortio Pod to be ready.

Verification: STATUS: Running with READY: 2/2 confirms fortio client is ready.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl get pods -l app=fortio
NAME                                READY   STATUS    RESTARTS   AGE
fortio-deploy-8f5fbbb64-mwnrcr      2/2     Running   0           22s
```

Set Environment Variable.

```
PS C:\Users\WangShiyu\istio-1.28.0> $FORTIO_POD = kubectl get pods -l app=fortio -o jsonpath='{.items[0].metadata.name}'
PS C:\Users\WangShiyu\istio-1.28.0> echo $FORTIO_POD
fortio-deploy-8f5fbbb64-mwnrcr
```

(4) Step 4: Test Normal Request

Send single request to test connection.

Verification: HTTP/1.1 200 OK confirms successful single request.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl exec $FORTIO_POD -c fortio -- /usr/bin/fortio curl -quiet http://httpbin:8000/get
HTTP/1.1 200 OK
access-control-allow-credentials: true
access-control-allow-origin: *
content-type: application/json; charset=utf-8
date: Sat, 16 May 2026 13:54:40 GMT
content-length: 621
x-envoy-upstream-service-time: 17
server: envoy

{
  "args": {},
  "headers": {
    "Host": [
      "httpbin:8000"
    ],
    "User-Agent": [
      "fortio.org/fortio-1.69.5"
    ],
    "X-Envoy-Attempt-Count": [
      "1"
    ],
    "X-Forwarded-Client-Cert": [
      "By=spiffe://cluster.local/ns/default/sa/httpbin;Hash=ea2dd6601fb8cf813b4a6b37213965d18339c823297d802db0948d23992f74a6;Subject=\"\";URI=spiffe://cluster.local/ns/default/sa/default"
    ],
    "X-Forwarded-Proto": [
      "http"
    ],
    "X-Request-Id": [
      "2ec94184-c56a-9c6e-a3d3-7a5ab7e015de"
    ]
  },
  "method": "GET",
  "origin": "127.0.0.6:59731",
  "url": "http://httpbin:8000/get"
}
```

(5) Step 5: Trigger Circuit Breaker (First Test)

Send concurrent requests (2 concurrent connections, 20 requests).

- Code 200 : 10 (50.0%): 10 requests succeeded
- Code 503 : 10 (50.0%): 10 requests rejected by circuit breaker

With maxConnections: 1 and http1MaxPendingRequests: 1, the connection pool can handle at most 2 concurrent requests (1 active + 1 pending). With 2 concurrent connections generating 20 total requests, 50% are rejected.

```

PS C:\Users\WangShiyu\istio-1.28.0> kubectl exec $FORTIO_POD -c fortio -- /usr/bin/fortio load -c 2 -qps 0 -n 20 -loglevel
Warning http://httpbin:8000/get
{"ts":1778939750.976811,"level":"info","r":1,"file":"logger.go","line":298,"msg":"Log level is now 3 Warning (was 2 Info
)"}
Fortio 1.69.5 running at 0 queries per second, 20->20 procs, for 20 calls: http://httpbin:8000/get
Starting at max qps with 2 thread(s) [gomax 20] for exactly 20 calls (10 per thread + 0)
{"ts":1778939750.982847,"level":"warn","r":37,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939750.986512,"level":"warn","r":36,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1778939750.989155,"level":"warn","r":37,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939750.992470,"level":"warn","r":36,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1778939750.994544,"level":"warn","r":37,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939750.995541,"level":"warn","r":36,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1778939750.99675,"level":"warn","r":37,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939750.997276,"level":"warn","r":37,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939750.998342,"level":"warn","r":36,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1778939750.999675,"level":"warn","r":37,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939751.000780,"level":"warn","r":36,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":0,"run":0}
Ended after 21.993769ms : 20 calls. qps=909.35

Aggregated Function Time : count 20 avg 0.0021333912 +/- 0.001881 min 0.000203182 max 0.005990367 sum 0.042667824
# range, mid point, percentile, count
>= 0.000203182 <= 0.001 , 0.000601591 , 40.00, 8
> 0.001 <= 0.002 , 0.0015 , 50.00, 2
> 0.002 <= 0.003 , 0.0025 , 75.00, 5
> 0.003 <= 0.004 , 0.0035 , 80.00, 1
> 0.004 <= 0.005 , 0.0045 , 85.00, 1
> 0.005 <= 0.00599037 , 0.00549518 , 100.00, 3
# target 50% 0.002
# target 75% 0.003
# target 90% 0.00533012
# target 99% 0.00592434
# target 99.9% 0.00598376
Error cases : count 10 avg 0.0007311022 +/- 0.0008618 min 0.000203182 max 0.00303751 sum 0.007311022
# range, mid point, percentile, count
>= 0.000203182 <= 0.001 , 0.000601591 , 80.00, 8
> 0.001 <= 0.002 , 0.0015 , 90.00, 1
> 0.003 <= 0.00303751 , 0.00301876 , 100.00, 1
# target 50% 0.000658507
# target 75% 0.000943084
# target 90% 0.002
# target 99% 0.00303376
# target 99.9% 0.00303713
# Socket and IP used for each connection:
[0] 5 socket used, resolved to 10.103.6.238:8000, connection timing : count 5 avg 0.000165469 +/- 9.67e-05 min 8.5474e
-05 max 0.000344497 sum 0.000827345
[1] 6 socket used, resolved to 10.103.6.238:8000, connection timing : count 6 avg 0.00012806683 +/- 7.96e-05 min 6.498
1e-05 max 0.000298281 sum 0.000768401
Connection time (s) : count 11 avg 0.00014506782 +/- 8.974e-05 min 6.4981e-05 max 0.000344497 sum 0.001595746
Sockets used: 11 (for perfect keepalive, would be 2)
Uniform: false, Jitter: false, Catchup allowed: true
IP addresses distribution:
10.103.6.238:8000: 11
Code 200 : 10 (50.0 %)
Code 503 : 10 (50.0 %)
Response Header Sizes : count 20 avg 122.5 +/- 122.5 min 0 max 245 sum 2450
Response Body/Total Sizes : count 20 avg 553.5 +/- 312.5 min 241 max 866 sum 11070
All done 20 calls (plus 0 warmup) 2.133 ms avg, 909.3 qps

```

(6) Step 6: Increase Load (Second Test)

Increase to 3 concurrent connections.

- Code 200 : 10 (33.3%): Success rate dropped
- Code 503 : 20 (66.7%): Rejection rate increased

With 3 concurrent connections (exceeding the pool limit of 2), the circuit breaker rejects 66.7% of requests.

```

PS C:\Users\WangShiyu\istio-1.28.0> kubectl exec $FORTIO_POD -c fortio -- /usr/bin/fortio load -c 3 -qps 0 -n 30 -loglev
el Warning http://httpbin:8000/get
{"ts":1778939827.330760,"level":"info","r":1,"file":"logger.go","line":298,"msg":"Log level is now 3 Warning (was 2 Info
)"}
Fortio 1.69.5 running at 0 queries per second, 20->20 procs, for 30 calls: http://httpbin:8000/get
Starting at max qps with 3 thread(s) [gomax 20] for exactly 30 calls (10 per thread + 0)
{"ts":1778939827.336391,"level":"warn","r":21,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1778939827.336536,"level":"warn","r":22,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939827.337790,"level":"warn","r":22,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939827.338619,"level":"warn","r":22,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939827.339286,"level":"warn","r":23,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":2,"run":0}
{"ts":1778939827.339785,"level":"warn","r":22,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939827.340599,"level":"warn","r":22,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939827.341942,"level":"warn","r":22,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939827.342177,"level":"warn","r":21,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1778939827.343640,"level":"warn","r":22,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939827.344190,"level":"warn","r":23,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":2,"run":0}
{"ts":1778939827.345271,"level":"warn","r":23,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":2,"run":0}
{"ts":1778939827.345493,"level":"warn","r":22,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939827.346856,"level":"warn","r":23,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":2,"run":0}
{"ts":1778939827.347922,"level":"warn","r":21,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1778939827.349542,"level":"warn","r":21,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1778939827.350050,"level":"warn","r":22,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":1,"run":0}
{"ts":1778939827.351055,"level":"warn","r":23,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":2,"run":0}
{"ts":1778939827.353408,"level":"warn","r":21,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":0,"run":0}
{"ts":1778939827.354969,"level":"warn","r":23,"file":"http_client.go","line":1151,"msg":"Non ok http code","code":503,"s
tatus":"HTTP/1.1 503","thread":2,"run":0}
Ended after 22.74409ms : 30 calls. qps=1319

```

```

Aggregated Function Time : count 30 avg 0.0019423499 +/- 0.001511 min 0.000284242 max 0.005141588 sum 0.058270498
# range, mid point, percentile, count
>= 0.000284242 <= 0.001 , 0.000642121 , 33.33, 10
> 0.001 <= 0.002 , 0.0015 , 70.00, 11
> 0.002 <= 0.003 , 0.0025 , 73.33, 1
> 0.003 <= 0.004 , 0.0035 , 86.67, 4
> 0.004 <= 0.005 , 0.0045 , 93.33, 2
> 0.005 <= 0.00514159 , 0.00507079 , 100.00, 2
# target 50% 0.00145455
# target 75% 0.003125
# target 90% 0.0045
# target 99% 0.00512035
# target 99.9% 0.00513946
Error cases : count 20 avg 0.001025435 +/- 0.0005506 min 0.000284242 max 0.001854936 sum 0.020508699
# range, mid point, percentile, count
>= 0.000284242 <= 0.001 , 0.000642121 , 50.00, 10
> 0.001 <= 0.00185494 , 0.00142747 , 100.00, 10
# target 50% 0.001
# target 75% 0.00142747
# target 90% 0.00168395
# target 99% 0.00183784
# target 99.9% 0.00185323
# Socket and IP used for each connection:
[0] 6 socket used, resolved to 10.103.6.238:8000, connection timing : count 6 avg 0.00014060017 +/- 4.612e-05 min 8.41
09e-05 max 0.000201185 sum 0.000843601
[1] 9 socket used, resolved to 10.103.6.238:8000, connection timing : count 9 avg 0.00016026556 +/- 5.941e-05 min 8.14
16e-05 max 0.000245313 sum 0.00144239
[2] 6 socket used, resolved to 10.103.6.238:8000, connection timing : count 6 avg 0.00019512867 +/- 4.083e-05 min 0.00
0128029 max 0.000243716 sum 0.001170772
Connection time (s) : count 21 avg 0.00016460776 +/- 5.51e-05 min 8.1416e-05 max 0.000245313 sum 0.003456763
Sockets used: 21 (for perfect keepalive, would be 3)
Uniform: false, Jitter: false, Catchup allowed: true
IP addresses distribution:
10.103.6.238:8000: 21
Code 200 : 10 (33.3 %)
Code 503 : 20 (66.7 %)
Response Header Sizes : count 30 avg 81.666667 +/- 115.5 min 0 max 245 sum 2450
Response Body/Total Sizes : count 30 avg 449.33333 +/- 294.6 min 241 max 866 sum 13480
All done 30 calls (plus 0 warmup) 1.942 ms avg, 1319.0 qps

```

(7) Step 7: Verify Envoy Statistics

View Envoy proxy statistics.

- remaining_pending: 1: 1 pending request slot available
- upstream_rq_pending_overflow: 30: 30 requests rejected due to pool overflow

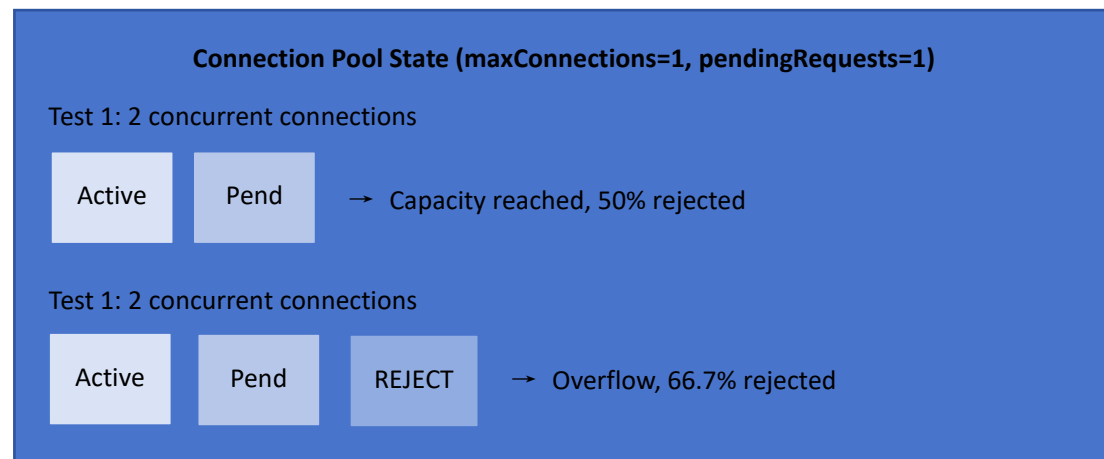
```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl exec $FORTIO_POD -c istio-proxy -- pilot-agent request GET stats | Select-String "httpbin.*pending"
cluster.outbound|8000||httpbin.default.svc.cluster.local;.circuit_breakers.default.remaining_pending: 1
cluster.outbound|8000||httpbin.default.svc.cluster.local;.circuit_breakers.default.rq_pending_open: 0
cluster.outbound|8000||httpbin.default.svc.cluster.local;.circuit_breakers.high.rq_pending_open: 0
cluster.outbound|8000||httpbin.default.svc.cluster.local;.upstream_rq_pending_active: 0
cluster.outbound|8000||httpbin.default.svc.cluster.local;.upstream_rq_pending_failure_eject: 0
cluster.outbound|8000||httpbin.default.svc.cluster.local;.upstream_rq_pending_overflow: 30
cluster.outbound|8000||httpbin.default.svc.cluster.local;.upstream_rq_pending_total: 21
```

(8) Step 8: Results Analysis

Circuit Breaker Effectiveness:

Test Scenario	Concurrent Connections	Success Rate	503 Rate
Baseline	1	100%	0%
Test 1	2	50%	50%
Test 2	3	33.3%	66.7%

Connection Pool Visualization:



(9) Step 9: Cleanup

Delete circuit breaking rule, httpbin service and fortio client.

```
PS C:\Users\WangShiyu\istio-1.28.0> kubectl delete destinationrule httpbin
destinationrule.networking.istio.io "httpbin" deleted
PS C:\Users\WangShiyu\istio-1.28.0> kubectl delete -f samples/httpbin/httpbin.yaml
serviceaccount "httpbin" deleted
service "httpbin" deleted
deployment.apps "httpbin" deleted
PS C:\Users\WangShiyu\istio-1.28.0> kubectl delete -f samples/httpbin/sample-client/fortio-deploy.yaml
service "fortio" deleted
deployment.apps "fortio-deploy" deleted
```

3. Conclusion

1) Summary of Achievements

This lab successfully demonstrated four essential Istio traffic management capabilities:

Capability	Resource Types Used	Key Outcome
Traffic Shifting	VirtualService, DestinationRule	Canary deployment with gradual traffic migration
Request Routing	VirtualService (match conditions)	Content-based routing for A/B

		testing
Fault Injection	VirtualService (fault field)	Resilience testing with delay and abort faults
Circuit Breaking	DestinationRule (connectionPool, outlierDetection)	Service protection with connection limits

2) Critical Observations

- **Traffic Shifting:** Probabilistic distribution at request level; small samples may deviate slightly from configured weights
- **Request Routing:** Rules evaluated sequentially; first match wins; default catch-all recommended
- **Fault Injection:** Revealed timeout misconfiguration issue (6s outer timeout vs 7s inner delay)
- **Circuit Breaking:** Immediate protection (503 responses) prevents cascading failures

3) Challenges and Solutions

Challenge	Solution
Image pull failures in China mainland	Used mirror registry hub.rat.dev
PowerShell syntax differences	Used here-strings (@"...") and Select-String
Timeout misconfiguration exposed	Documented as learning point for timeout coordination